

ACCESO A DATOS (AD)

lridruejo

DAM

2025 / 2026

ÍNDICE

UNIDAD 1: Manejo de ficheros.....	1
1. Clases asociadas a las operaciones de gestión de ficheros y directorios.....	2
CLASE FILE	2
INTERFAZ FILENAMEFILTER	2
RUTAS DE LOS FICHEROS.....	3
2. Flujos.	4
FLUJOS BASADOS EN BYTES	4
FLUJOS BASADOS EN CARACTERES	5
3. Formas de acceso a un fichero.	6
OPERACIONES CON ARCHIVOS SECUENCIALES	6
OPERACIONES CON ARCHIVOS ALEATORIOS.....	6
4. Java NIO.....	7
JAVA NIO PATH	7
JAVA NIO FILES.....	8
ESCRIBIR CONTENIDO EN UN FICHERO.....	8
5. Trabajo con ficheros XML: analizadores sintácticos (parser) y vinculación (binding).	10
JAXB	10
PARSER O ANALIZADOR XML	11
DOM	11
SAX	13
STAX.....	13
BINDING	14
6. Librerías para la conversión de documentos XML a otros formatos.....	16
JASPERREPORTS	16
UNIDAD 2: Manejo de conectores.....	17
DESFASE OBJETO-RELACIONAL.....	17
1. Protocolos de acceso a bases de datos.....	17
2. Conectores o drivers.	19
3. Arquitectura JDBC.	20
TIPO 1 (PUENTE JDBC-ODBC)	20
TIPO 2 (API NATIVA)	20
TIPO 3 (NET PURE JAVA)	21
TIPO 4 (PROTOCOLO NATIVO)	21
UNIDAD 3: Herramientas de mapeo objeto-relacional (ORM).	22
UNIDAD 4: Bases de datos objeto-relacionales y orientadas a objetos.	23
1. Características de las BDOO.	23
VENTAJAS	23

DESVENTAJAS	24
2. Gestores de BDOO.....	25
OBJETOS SIMPLES Y ESTRUCTURADOS	25
3. El API de la base de datos.....	26
APERTURA Y CIERRE DE CONEXIONES.....	26
4. El lenguaje de consulta de objetos OQL.	27
CONSULTAS SELECT	27
5. Características de las BDOR.....	28
6. Gestores de BDOR.	29
TIPOS DE OBJETOS.....	29
7. Gestión de transacciones en las BDOR.....	31
UNIDAD 5: Bases de datos NoSQL.	32
1. Bases de Datos Nativas XML.	33
ESTRATEGIAS DE ALMACENAMIENTO.....	33
COLECCIONES Y DOCUMENTOS	34
GESTORES NATIVOS XML COMERCIALES Y LIBRES.....	34
2. Base de datos eXist.	36
LENGUAJE DE CONSULTA XPATH	36
LENGUAJE DE CONSULTAS XQUERY.....	36
3. Bases de Datos MongoDB.	37

UNIDAD 1: Manejo de ficheros.

Al apagar un ordenador, los datos de la memoria RAM se pierden. Los ordenadores utilizan ficheros para almacenar información de forma persistente en soportes de almacenamiento como discos duros, DVD o pendrives. Los datos persistentes mantienen su información más allá de la ejecución de una aplicación.

Las operaciones de **Entrada/Salida (E/S)** gestionan el flujo de información entre un programa y el exterior.

Java proporciona el paquete **java.io** para realizar operaciones con ficheros, incluyendo interfaces, clases y excepciones:

- Leer datos desde flujos de entrada.
- Escribir datos en flujos de salida.
- Operar con ficheros en el sistema local.
- Gestionar la serialización de objetos.

java.nio (Non-Blocking I/O) se introdujo en la versión 1.4 como una alternativa más eficiente, basada en "Buffer" y "Channel".

Comparación entre java.io y java.nio:

- Ambos paquetes pueden utilizarse de forma complementaria.
- java.nio permite manejar múltiples canales con pocos hilos.
- El procesamiento de datos en java.nio es más complejo que con los streams de java.io.
- java.nio es adecuado para muchas conexiones con poco tráfico de datos.
- java.io es preferible para pocas conexiones con alto volumen de datos.

Hasta JSE7, la clase **java.io.File** se utilizaba para operaciones de E/S, pero presentaba limitaciones como:

- Métodos que no lanzaban excepciones, dificultando la gestión de errores.
- Incapacidad para manejar enlaces simbólicos.
- Problemas de escalabilidad con directorios largos.

El paquete **java.nio.file**, incorporado en JSE7, resuelve estos problemas y es la opción recomendada para trabajar con archivos, independientemente del método de E/S utilizado.

1. Clases asociadas a las operaciones de gestión de ficheros y directorios.

CLASE FILE

La clase File ofrece una representación abstracta e independiente de la plataforma para archivos y directorios. Sus instancias representan nombres de archivo, no los archivos físicos. Esta clase permite examinar atributos, realizar operaciones de mantenimiento y navegar por la estructura de directorios del sistema de archivos.

Funcionalidades principales:

- Renombrar archivos con `renameTo()`.
- Borrar archivos con `delete()` o `deleteOnExit()`.
- Crear ficheros temporales con `createTempFile()`.
- Establecer fechas de modificación con `setLastModified()`.
- Crear directorios con `mkdir()` o `mkdirs()`.
- Listar contenido de directorios con `list()` y `listFiles()`.
- Obtener raíces del sistema con `listRoots()`.

Constructores:

- `File(String nombreFichero|path).`
- `File(String path, String nombreFichero|path).`
- `File(File path, String nombreFichero|path).`

La creación de un objeto File no verifica la existencia del archivo ni crea archivos automáticamente. Las rutas pueden ser absolutas o relativas.

Métodos destacados:

- **Verificación** → `canRead()`, `canWrite()`, `exists()`, `isDirectory()`, `isFile()`.
- **Información** → `getName()`, `getPath()`, `getAbsolutePath()`, `length()`, `lastModified()`.
- **Operaciones** → `createNewFile()`, `delete()`, `mkdir()`, `renameTo()`.
- **Navegación** → `list()`, `getParent()`, `getParentFile()`.

INTERFAZ FILENAMEFILTER

La interfaz FilenameFilter permite filtrar archivos según criterios específicos del nombre. Una clase que implemente esta interfaz debe definir el método:

`boolean accept(File dir, String nombre)`

Este método devuelve true cuando el archivo cumple con los criterios de filtrado establecidos, permitiendo mostrar solo los archivos que coincidan con patrones o condiciones específicas en su nombre.

RUTAS DE LOS FICHEROS

El carácter separador de directorios varía según el sistema operativo. Para garantizar la portabilidad de las aplicaciones Java, se recomienda utilizar **File.separator** en lugar de caracteres específicos como "/" o "".

Esta constante proporciona automáticamente el separador correcto para el sistema donde se ejecuta el programa. Se puede crear una función que adapte cualquier ruta utilizando File.separator para asegurar compatibilidad multiplataforma.

2. Flujos.

Los programas Java realizan operaciones de Entrada/Salida mediante **flujos o streams**. Un stream representa una abstracción para producir o consumir información, donde el sistema de E/S de Java se encarga de vincularlo con el dispositivo físico correspondiente.

Java define dos tipos de flujos en el paquete java.io:

- ▶ **Byte streams (8 bits):**
 - Gestionan entradas y salidas de bytes.
 - Orientados a datos binarios.
 - Clases abstractas: **InputStream** y **OutputStream**.
 - Métodos principales: **read()** y **write()** para bytes.
- ▶ **Character streams (16 bits):**
 - Manejan flujos de caracteres Unicode.
 - Clases abstractas: **Reader** y **Writer**.
 - Métodos principales: **read()** y **write()** para caracteres.

Las clases y métodos de E/S son consistentes independientemente del dispositivo físico, liberando al programador de conocer los detalles específicos de cada dispositivo.

FLUJOS BASADOS EN BYTES

Para el manejo de flujos de bytes, Java utiliza las **clases abstractas **InputStream** y **OutputStream****. Los archivos binarios almacenan datos en formato UTF-8, representando la información en código binario.

Clases principales que **heredan de **OutputStream** (escritura)**:

- **FileOutputStream** → Escribe bytes en archivos. Sobrescribe el contenido existente a menos que se use el parámetro append: true.
- **ObjectOutputStream** → Convierte objetos y variables en secuencias de bytes para su almacenamiento.
- **DataOutputStream** → Formatea tipos primitivos y Strings con métodos como **writeByte()**, **writeFloat()**.

Clases principales que **heredan de **InputStream** (lectura)**:

- **FileInputStream** → Lee bytes directamente desde archivos.
- **ObjectInputStream** → Reconstruye objetos y variables a partir de secuencias de bytes leídas.

Estas clases permiten la manipulación de datos binarios manteniendo la consistencia en la representación de la información.

FLUJOS BASADOS EN CARACTERES

Los flujos de caracteres en Java utilizan las clases abstractas **Reader** y **Writer**. Los accesos directos a disco con **FileReader** o **FileWriter** son ineficientes cuando se procesan pequeños fragmentos de datos, debido a los múltiples accesos físicos al disco.

Las clases con **buffer** (**BufferedReader**, **BufferedWriter**, **BufferedInputStream**, **BufferedOutputStream**) solucionan este problema mediante un almacenamiento intermedio. Al escribir, acumulan datos hasta tener suficiente cantidad para una escritura eficiente. Al leer, cargan más datos de los solicitados y los sirven desde el buffer en operaciones posteriores.

BufferedReader además proporciona funcionalidad para leer líneas completas, algo no disponible en **FileReader**. Es posible crear un **BufferedReader** a partir de un **FileReader** para combinar el acceso al archivo con la eficiencia del buffer.

3. Formas de acceso a un fichero.

En Java existen dos tipos principales de acceso a archivos:

- **Acceso secuencial** → Lectura o escritura de datos en orden, desde el inicio hasta el final.
- **Acceso aleatorio** → Permite acceder a cualquier posición del archivo directamente, requiriendo que el archivo esté completamente disponible.

A veces, se distingue una tercera forma:

- **Concatenación (tuberías o pipes)** → Son una forma de comunicación entre programas que se ejecutan simultáneamente, donde la salida de un programa se conecta directamente a la entrada de otro.

OPERACIONES CON ARCHIVOS SECUENCIALES

Las operaciones básicas sobre ficheros de acceso secuencial son crear/abrir, leer, borrar, copiar, buscar información y cerrar archivos.

La última lectura genera **EOFException**, que se captura para gestionar el cierre adecuado del archivo.

Clases recomendadas para archivos de texto:

- **Reader y Writer** → Optimizadas para caracteres Unicode (2 bytes por carácter).
- **FileReader** → Subclase de Reader para la lectura de archivos de texto.
- **FileWriter** → Subclase de Writer para la escritura en archivos de texto.

Bufferización para mejor rendimiento:

- **BufferedReader** → Buffer sobre FileReader.
- **BufferedWriter** → Buffer sobre FileWriter.

OPERACIONES CON ARCHIVOS ALEATORIOS

La clase **RandomAccessFile** combina lectura y escritura en una sola clase, requiere especificar el modo de acceso ("r" para solo lectura o "rw" para lectura/escritura), e incluye métodos de posicionamiento:

- **seek(long posicion)** → Va directamente a una posición específica.
- **skipBytes(int desplazamiento)** → Salta bytes desde la posición actual.

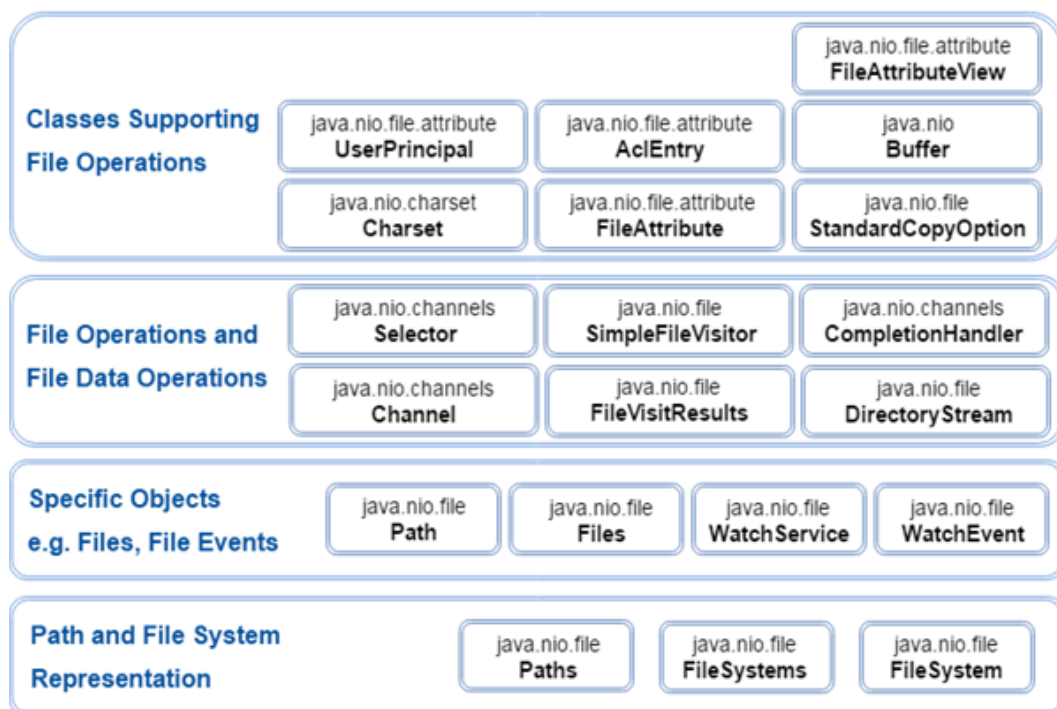
Los registros deben tener un tamaño fijo para un acceso predecible.

4. Java NIO.

Java NIO (New I/O), introducido en Java 7, es una API moderna que mejora y simplifica el manejo de entrada/salida frente a la API tradicional `java.io`. Sus ventajas principales son:

- **Canales y búferes** → En lugar de trabajar con flujos de datos (secuencias de bytes o de caracteres), NIO utiliza canales (para la conexión) y búferes (para contener los datos), lo que permite una gestión más eficiente.
- **E/S sin bloqueo** → Permite que un hilo realice otras tareas mientras un canal lee o escribe datos en un búfer, mejorando el rendimiento y la escalabilidad.
- **Selectores** → Un selector permite que un solo hilo gestione múltiples canales de manera eficiente, monitoreando eventos en ellos (como conexiones o datos recibidos).

Java NIO ofrece un modelo de E/S de alto rendimiento, más flexible y menos complejo que la E/S estándar, especialmente útil para aplicaciones que requieren manejar muchas conexiones simultáneamente.



JAVA NIO PATH

La interfaz **`java.nio.file.Path`** representa una ruta en el sistema de archivos y es la evolución moderna para reemplazar la antigua clase `File`. Características clave:

- **Propósito** → Localizar archivos o directorios, que pueden ser rutas absolutas (completas desde la raíz) o relativas (desde otra ruta).

- **Creación** → Se instancia fácilmente usando métodos estáticos de la clase `Paths.get()` a partir de un String.
- **Flexibilidad** → Permite trabajar con rutas sin que los archivos o directorios existan físicamente.
- **Funcionalidad** → Incluye métodos útiles para manipular rutas (obtener nombres, directorios padres, resolver rutas relativas...).
- **Dependencia del SO** → La implementación refleja el sistema de archivos del sistema operativo subyacente, lo que dificulta la comparación directa entre rutas de diferentes SO.

Además, la clase **FileSystem** representa el sistema de archivos completo, mientras que **Path** representa un elemento individual dentro de él. Juntas permiten realizar operaciones avanzadas de gestión de archivos.

JAVA NIO FILES

La clase **java.nio.file.Files** es el punto de entrada principal para manipular archivos y directorios reales en el disco. Características clave:

- **Métodos estáticos** → Todos sus métodos son estáticos y operan sobre objetos **Path**.
- **Operaciones esenciales** → Permite realizar las operaciones básicas de gestión de archivos: verificar existencia y accesibilidad, y borrar, copiar y mover archivos o directorios.

Para el borrado, hay dos métodos:

- **delete(Path)** → Borra y lanza una excepción si falla (si el archivo no existe, lanza `NoSuchFileException`).
- **deleteIfExists(Path)** → Borra y no lanza error si el archivo o directorio no existe.

Para el copiado de archivos o directorios, se utiliza el método **copy(Path, Path, CopyOption...)**. Por defecto, falla si el archivo destino ya existe (a menos que se use la opción **REPLACE_EXISTING**). Al copiar directorios, solo se copia el directorio en sí, no su contenido (el resultado es un directorio vacío).

ESCRIBIR CONTENIDO EN UN FICHERO

El parámetro **OpenOptions**, implementado comúnmente con el **enum StandardOpenOptions**, permite controlar cómo se accede a un archivo en Java, restringiendo los permisos más allá de los del sistema operativo para prevenir errores. Los valores principales de **modo de acceso** son:

- **WRITE** → Habilita la escritura.
- **APPEND** → Añade datos al final del archivo.
- **CREATE_NEW** → Crea un archivo nuevo y falla si ya existe.
- **CREATE** → Abre el archivo si existe o lo crea si no.

- **TRUNCATE_EXISTING** → Si el archivo existe, borra su contenido para sobrescribirlo.

Hay dos métodos de escritura:

- **Escritura desde arrays de bytes** → Es la forma más simple y se realiza usando **Files.write()**, pero es limitada.
- **Escritura desde buffers** → Es mucho más eficiente para archivos grandes. Utiliza clases como **BufferedWriter** y **BufferedReader** para escribir y leer línea a línea, optimizando el rendimiento.

5. Trabajo con ficheros XML: analizadores sintácticos (parser) y vinculación (binding).

XML es un metalenguaje creado para garantizar la interoperabilidad en el intercambio de datos entre diferentes plataformas y redes. Sus principales ventajas son:

- **Estándar universal** → Proporciona un formato y método de acceso estandarizados, ampliamente aceptado.
- **Estructura clara** → Organiza la información de manera jerárquica y definida, facilitando su posterior procesamiento y uso local por distintas aplicaciones.

Desde el punto de vista de un desarrollador, trabajar con XML implica un proceso de tres pasos:

- **Leer** un archivo de texto que sigue el estándar XML.
- **Cargar** la información en memoria para su procesamiento.
- **Procesar** los datos para obtener resultados (que a menudo se guardan en otro XML).

Este proceso se conoce como "**parsing**" (**análisis léxico-sintáctico**), realizado por programas llamados "**parsers**". Un parser XML se encarga de leer y analizar el documento para verificar que esté correctamente estructurado ("bien formado") y luego entregar su contenido a la aplicación.

Además, un **schema** (esquema) es una especificación en XML que define las reglas de un documento: qué elementos y atributos están permitidos, en qué orden y sus relaciones. Un documento que cumple con las reglas de su esquema se considera válido.

JAXB

JAXB (Java Architecture for XML Binding) es una tecnología de Java que simplifica el trabajo con XML al convertir automáticamente elementos XML en objetos Java y viceversa.

El objetivo principal es hacer **binding** o vincular un esquema XML a representaciones en código Java (generar un conjunto de clases que representan el esquema). Esto evita tener que manipular el XML manualmente.

Procesos fundamentales:

- **Compilador de esquema o schema compiler** → Genera las clases Java que representan el esquema XML.
- **Unmarshalling** → La aplicación, en tiempo de ejecución, convierte datos XML en un árbol de objetos Java (para leer y operar con los datos). Es

decir, convierte un documento XML específico en instancias de las clases Java creadas en la compilación.

- **Procesamiento y manipulación** → Se trabaja directamente sobre el código Java.
- **Marshalling** → Se codifican los objetos Java de nuevo en un archivo XML (para guardar los datos modificados o nuevos).

Las operaciones de marshalling y unmarshalling las proporciona el **binding runtime framework (framework de ejecución)**.

PARSER O ANALIZADOR XML

El XML es un lenguaje de marcado estandarizado que estructura información mediante etiquetas, similar a un contenedor que puede incluir otras etiquetas o texto. Al ser totalmente textual, soluciona los problemas de compatibilidad entre plataformas, ya que cualquier dato se representa como texto. Su estructura jerárquica es comparable a la de los objetos en la programación orientada a objetos.

Para leer XML, existen analizadores o "parsers", que son herramientas que validan y extraen la información del documento. Se clasifican en:

- **Analizadores secuenciales (SAX, StAX)** → Procesan el documento línea por línea, son rápidos pero ineficientes para accesos repetitivos, ya que requieren releer el documento.
- **Analizadores jerárquicos (DOM)** → Almacenan toda la estructura del XML en la memoria RAM como un árbol, lo que permite un acceso rápido y eficiente a cualquier parte de los datos. Siguen el estándar **DOM (Document Object Model)**, popularizado también por HTML y JavaScript.

Los parsers secuenciales son ideales para lectura simple y rápida, mientras que los jerárquicos son mejores para aplicaciones que necesitan consultar y manipular los datos repetidamente.

DOM

El **DOM (Document Object Model)** es un estándar del **W3C (World Wide Web Consortium)** que define una interfaz para manipular (leer y modificar el estilo, la estructura y el contenido) documentos XML. Para trabajar con DOM en Java se utilizan:

- ▶ **Lectura de XML** → Análisis o parsing para crear un DOM.
 - **DocumentBuilder** → Clase abstracta para construir el DOM desde un archivo XML.
 - **DocumentBuilderFactory** → Clase para poder crear instancias de DocumentBuilder.
 - **Document** → Clase que representa el DOM cargado en memoria.
- ▶ **Escritura de XML:**

- **Transformer** → Clase abstracta para convertir el DOM a texto XML.
- **TransformerFactory** → Clase para poder crear instancias de Transformer.
- **Source (fuente) / Result (destino)** → Adaptadores para trabajar con diferentes formatos (DOM, SAX, Stream).
- **DOMSource y StreamResult** → Clases para transformar DOM a archivos XML.

El DOM representa un documento XML como un árbol de nodos, donde cada componente (documento, elementos, texto, atributos) se convierte en un tipo específico de nodo con sus propios métodos para acceder a datos.

El DOM incluye información no visible como retornos de carro, que se interpretan como nodos hijos. El contenido textual de una etiqueta es un nodo hijo de tipo texto, por lo que para acceder al texto de una etiqueta hay que obtener su primer hijo.

La interfaz Document proporciona métodos para seleccionar partes del árbol (como elementos específicos) sin necesidad de recorrer todos los nodos, facilitando la navegación y extracción de información del XML.

Interfaces DOM:

- **Node** → Cualquier nodo del documento.
- **Document** → Nodo raíz que representa todo el documento, árbol DOM.
- **Element** → Nodos que corresponden a las etiquetas XML.
- **Text** → Contenido textual dentro de las etiquetas (de un elemento o atributo).
- **Attr** → Atributos de los elementos.
- **NodeList** → Colección de nodos a los que se accede con un índice.

Para navegar por el árbol se usan métodos como `getDocumentElement()`, `getElementsByTagName(etiqueta)`, `getParentNode()`, `getChildNodes()`, `getNodeValue()`, `getFirstChild()`, `getLastChild()`, `getNextSibling()`, `getPreviousSibling()`, `getAttribute(attrName)`...

Además de navegar por los nodos del árbol, existen métodos para añadir nuevos hijos (`appendChild()`), asignar el valor a un atributo, crear nuevos elementos o etiquetas (`createElement()`), contenido textual (`createTextNode()`) o comentarios (`createComment()`).

SAX

SAX (Simple API for XML) es un parser que procesa documentos XML de forma secuencial y basada en eventos, a diferencia de DOM que carga todo el árbol en memoria, lo que lo hace más eficiente. Es ideal para archivos grandes, ya que no carga el documento completo.

Recorre el XML **secuencialmente** (procesa el documento rápidamente en una sola pasada) y dispara eventos al encontrar partes del documento (como el inicio o fin de una etiqueta).

No permite navegar o modificar la estructura del XML, ya que solo accede a una parte a la vez y no ofrece una visión global del documento. Es útil cuando solo se necesita extraer información específica sin modificar la estructura y para parsear diferentes partes del documento.

SAX genera un objeto procesador de XML llamado **XMLReader**, que notifica eventos a objetos que implementan estas **interfaces**:

- **ContentHandler** → Gestiona los eventos principales del documento.
- **DTDHandler** → Maneja eventos relacionados con la **DTD (Document Type Definition)**.
- **ErrorHandler** → Gestiona los errores del parser.
- **EntityResolver** → Resuelve referencias a entidades externas.

La clase **DefaultHandler** implementa por defecto las interfaces y permite definir solo los métodos necesarios, facilitando la creación del parser. Sus eventos básicos son **startDocument**, **endDocument**, **startElement**, **endElement** y **Characters**.

STAX

StAX es una API Java para procesar XML que utiliza un modelo **PULL (la aplicación solicita la información cuando la necesita)**, a diferencia de SAX que es PUSH (notifica a la aplicación).

Permite tanto leer como escribir documentos XML y ofrece dos APIs diferentes:

- **Cursor API** → Usa un cursor que solo avanza (no retrocede) y no conserva información de eventos anteriores, por lo que es más eficiente en memoria.
- **Iterator API** → Representa el documento como un flujo de eventos discretos y permite navegar hacia atrás en la jerarquía, por lo que proporciona más información y flexibilidad.

Procesa los documentos XML de arriba abajo, procesando los tokens secuencialmente y generando eventos durante el análisis.

Se debe utilizar StAX cuando el documento no está profundamente anidado y cuando el DOM consumiría demasiada memoria por ser un documento muy grande. También es útil cuando se necesita procesar solo parte de un documento o streams en tiempo real.

Los **eventos** más importantes son **StartDocument**, **EndDocument**, **StartElement**, **EndElement**, **Characters** y **Comments**:

- Los elementos vacíos también tienen sus eventos de inicio y fin.
- Los atributos se manejan como eventos secundarios y se accede a ellos desde el evento de inicio del elemento correspondiente.
- Los namespaces también se manejan como eventos secundarios, pero aparecen dos veces y se accede a ellos desde los eventos de inicio y de fin del elemento correspondiente. Además, se guardan en una pila a la que se puede acceder con la interfaz **javax.xml.namespace.NamespaceContext**.
- Los eventos de caracteres se especifican para todos los elementos y se pueden dividir.

BINDING

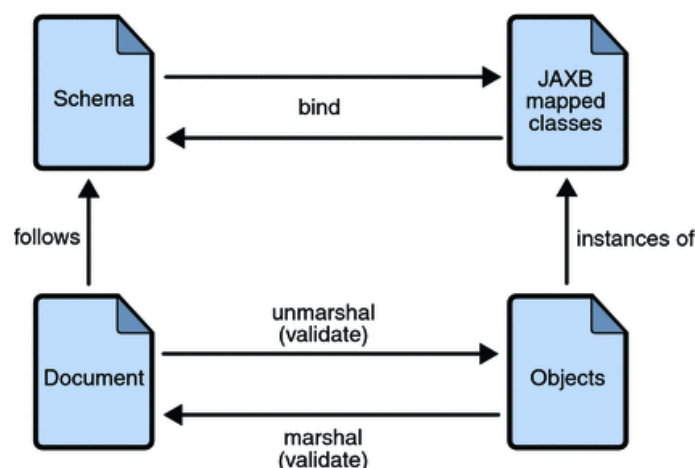
El **binding** es una técnica que automatiza la vinculación entre clases Java y formatos de almacenamiento específicos.

En Java, una de las bibliotecas más conocidas para este propósito es **JAXB (Java Architecture for XML Binding)**, que permite mapear clases Java a XML y viceversa. JAXB fue incluido en el JDK desde la versión 6 hasta la 10, pero se eliminó a partir del JDK 11. Para utilizarlo en versiones posteriores, es necesario agregar las dependencias correspondientes en el archivo **pom.xml**.

JAXB ofrece dos funcionalidades principales:

- **Serialización** → Convertir objetos Java en XML.
- **Deserialización** → Reconstruir objetos Java a partir de XML.

Esto elimina la necesidad de implementar manualmente rutinas para guardar o cargar datos en formato XML.



Las **anotaciones** en Java son clases especiales (precedidas por @) que pueden asociarse a un paquete, una clase, un atributo o un parámetro. Se inyectan en elementos del código durante la compilación. No afectan al modelo de datos, pero pueden ser usadas por aplicaciones que las necesiten. Pueden tener parámetros (otras anotaciones, constantes) disponibles en tiempo de compilación. Las principales anotaciones son:

- **@XmlRootElement** → Define la clase como la raíz del documento XML. Se escribe sobre la definición de la clase.
- **@XmlType** → Especifica el orden de los elementos en el XML (para clases que no son la raíz).
- **@XmlElement** → Mapea un atributo a una etiqueta XML. Es opcional si el nombre coincide. Se escribe antes del setter.
- **@XmlAttribute** → Mapea un atributo a un nodo XML.
- **@XmlElementWrapper** → Crea una etiqueta contenedora para colecciones de elementos.

Todas estas anotaciones suelen incluir el parámetro **name** para definir el nombre personalizado de la etiqueta o atributo XML.

JAXB convierte automáticamente **esquemas XML (XSD, DTD, WSDL)** en **clases Java (xjc)** y viceversa (**schemagen**). Las clases Java se generan con anotaciones para definir el mapeo, usando **java.util.List** para las secuencias. Para trabajar con los datos, se utiliza una instancia de **JAXBContext** para crear los serializadores y deserializadores.

6. Librerías para la conversión de documentos XML a otros formatos.

JASPERREPORTS

JasperReports es una librería de código abierto y gratuita (bajo licencia GPL) que solucionó la generación de informes en Java. Escrita completamente en Java, se integra como un archivo JAR en cualquier IDE.

Su estructura de carpetas incluye:

- **build** → Las clases de la librería sin empaquetar.
- **demo** → Ejemplos de uso listos para compilar con Ant.
- **dist** → La librería empaquetada (JAR) y la documentación Javadoc.
- **docs** → Referencia rápida en XML.
- **lib** → Otras librerías necesarias para funcionalidades como gráficos o exportación.
- **src** → Los archivos fuente del código.

Los pasos para trabajar con JasperReports son:

- ▶ **Diseñar la plantilla (.jrxml)** → Se crea un archivo XML que sigue una estructura DTD definida por JasperReports. Este se organiza en secciones:
 - **title** → Título del informe.
 - **pageHeader** → Encabezado del informe.
 - **columnHeader** → Encabezado de las columnas.
 - **detail** → Cuerpo del documento.
 - **columnFooter** → Pie de la columna.
 - **pageFooter** → Pie de página.
 - **summary** → Cierre del documento.
- ▶ **Compilación** → El diseño XML se compila usando el método **compileReport()**. Este proceso convierte la plantilla en un objeto serializable de tipo **net.sf.jasperreports.engine JasperReport**, listo para ser llenado con datos.
- ▶ **Llenar el informe con datos** → Se utiliza el método **fillReportXXX()**, al que se le pasa el diseño compilado y una conexión a la base de datos (u otra fuente de datos). El resultado es un objeto **JasperPrint**, que representa el documento final listo para usar. Este objeto puede guardarse o procesarse inmediatamente.
- ▶ **Visualización y exportación** → El objeto JasperPrint se puede:
 - Ver en pantalla usando la clase **JasperViewer**.
 - Imprimir usando los métodos de la clase **JasperPrintManager**.
 - Exportar a otros formatos usando los métodos **exportReportXXX()**.

UNIDAD 2: Manejo de conectores.

Las bases de datos relacionales son el sistema de almacenamiento más extendido, organizando la información en tablas compuestas por filas y columnas. Cada tabla representa una colección de objetos del mismo tipo, y utiliza una clave primaria para identificar de forma única cada registro.

El **DBMS (Database Management System o Sistema Gestor de Bases de Datos, SGBD)** es el software que se encarga de todas las operaciones (administrar el almacenamiento, mantenimiento y recuperación) con los datos. Cuando se trata de bases de datos relacionales, este sistema se denomina **RDBMS (Relational Database Management System)**.

Antes, la programación con bases de datos era fragmentada, ya que cada SGBD utiliza su propio lenguaje de comunicación.

Java solucionó esta fragmentación con **JDBC (Java Database Connectivity)**, una API que simplifica el acceso a bases de datos relacionales unificando la comunicación entre aplicaciones Java y diversos motores de bases de datos.

DESFASE OBJETO-RELACIONAL

El **desfase objeto-relacional** es la incompatibilidad entre el modelo orientado a objetos (POO) y las bases de datos relacionales, lo que genera dificultades técnicas al guardar objetos en tablas. Esto es debido a que:

- El desarrollador debe dominar tanto un lenguaje de POO como SQL.
- Las bases de datos relacionales, al tratar con relaciones y conjuntos debido a su naturaleza matemática, tienen tipos limitados, mientras que la POO maneja estructuras complejas (objetos y las asociaciones entre ellos).
- Requiere traducir un modelo de objetos (clases, asociaciones) a un modelo relacional (tablas, filas), lo que implica diseñar diagramas distintos para los distintos paradigmas.

La **impedancia objeto-relacional** es el conjunto de dificultades técnicas que surgen cuando una base de datos relacional se usa en conjunto con un programa escrito en POO.

1. Protocolos de acceso a bases de datos.

Inicialmente, los SGBD usaban soluciones propietarias. La aparición de **ODBC (Open Database Connectivity)** estandarizó la conexión a bases de datos desde diferentes lenguajes, pero su complejidad limitó su adopción fuera de Windows y en el mundo Java.

Para solucionarlo, se creó JDBC, una API específica para Java que es:

- Similar a ODBC en funcionalidad, pero diseñada bajo los principios de Java (orientada a objetos y multiplataforma).
- Sencilla y flexible, permitiendo construir y ejecutar sentencias SQL.
- Independiente de la base de datos, aunque puede usar la sintaxis específica de cada SGBD.

JDBC funciona de la siguiente manera:

- La aplicación Java solicita una conexión usando una URL específica (por ejemplo, jdbc:oracle:thin:@localhost:1521:XE).
- La clase Java DriverManager localiza y utiliza el controlador (driver) JDBC adecuado para esa URL.
- El controlador establece la conexión con la base de datos y, si tiene éxito, devuelve un objeto Connection a la aplicación.
- Si falla, lanza una SQLException.

JDBC actúa como una capa intermedia que permite a las aplicaciones Java comunicarse con cualquier base de datos de manera uniforme, gracias a los controladores proporcionados por los desarrolladores de cada SGBD.

JDBC ofrece clases e interfaces para establecer una conexión a una base de datos, ejecutar una consulta y procesar los resultados.

2. Conectores o drivers.

Un **driver o conector JDBC** es un archivo (normalmente **.jar**) que implementa la interfaz del API JDBC y permite a una aplicación Java conectarse y trabajar con una base de datos específica.

El driver, proporcionado por el fabricante de la base de datos o por un tercero, actúa como traductor, permitiendo que la aplicación se comuniquen con la base de datos. El código de la aplicación usa los paquetes **java.sql** y **javax.sql**, por lo que no depende del driver. JDBC se encarga de ocultar los detalles de la base de datos.

El driver es el componente esencial que hace posible la conexión, mientras que JDBC proporciona las herramientas uniformes para interactuar con cualquier base de datos desde el código.

Todos los controladores o conectores JDBC deben ser compatibles, como mínimo, con el nivel básico (**Entry Level**, una lista específica de capacidades de SQL) del estándar **ANSI SQL-2** (el estándar establecido en **1992** por el **American National Standards Institute**). Esto garantiza una funcionalidad SQL fundamental y uniforme en todos los drivers. Los desarrolladores son quienes certifican que sus conectores cumplen con este estándar.

3. Arquitectura JDBC.

Existen dos modelos de arquitectura para conectar una aplicación con una base de datos:

- ▶ **Modelo de dos capas (Cliente/Servidor)** → La aplicación (cliente) se comunica directamente con la fuente de datos (servidor), que puede estar en una máquina diferente conectada por red, utilizando un conector JDBC específico para esa base de datos.
- ▶ **Modelo de tres capas** → La aplicación envía los comandos a una capa intermedia de servicios, que es la que se comunica con la fuente de datos y envía los resultados al usuario.

Hoy en día hay cinco tipos de controladores JDBC en uso:

- **Tipo 1** → Puente JDBC-ODBC (obsoleto).
- **Tipo 2** → Controlador parcial de Java.
- **Tipo 3** → Controlador Java puro para middleware de base de datos.
- **Tipo 4** → Controlador Java puro para directo a base de datos.
- **Tipo 5** → Controladores altamente funcionales con un rendimiento superior.

Los drivers tipo 1 y tipo 2 utilizan código nativo vía **JNI (Java Native Interface)**, que los hace eficientes pero dependientes del software cliente.

TIPO 1 (PUENTE JDBC-ODBC)

Este conector actúa como un puente que traduce las llamadas de JDBC a ODBC para interactuar con una base de datos.

No requiere un driver ODBC específico para cada base de datos y es universal, está soportado por muchos fabricantes, permitiendo acceso a muchas bases de datos.

Sin embargo, no está disponible en todas las plataformas, tiene un bajo rendimiento debido a las múltiples conversiones (JDBC → ODBC → Base de datos) y requiere configuración manual del **DSN (Data Source Names)** en el gestor de ODBC.

TIPO 2 (API NATIVA)

Este conector convierte las llamadas JDBC directamente a las funciones específicas de la base de datos (como Oracle o SQL Service).

Ofrece un mejor rendimiento que el tipo 1 porque se comunica directamente con el servidor, pero requiere instalar la librería nativa del fabricante en cada máquina cliente, lo que impide su uso para **aplicaciones web o Internet**.

TIPO 3 (NET PURE JAVA)

Es un driver basado en servidor que utiliza una **arquitectura de tres capas**. En lugar de conectarse directamente, el cliente se comunica con un servidor **middleware** a través de la red. Este servidor intermedio es el que realiza la traducción de protocolos y la conexión real con la base de datos.

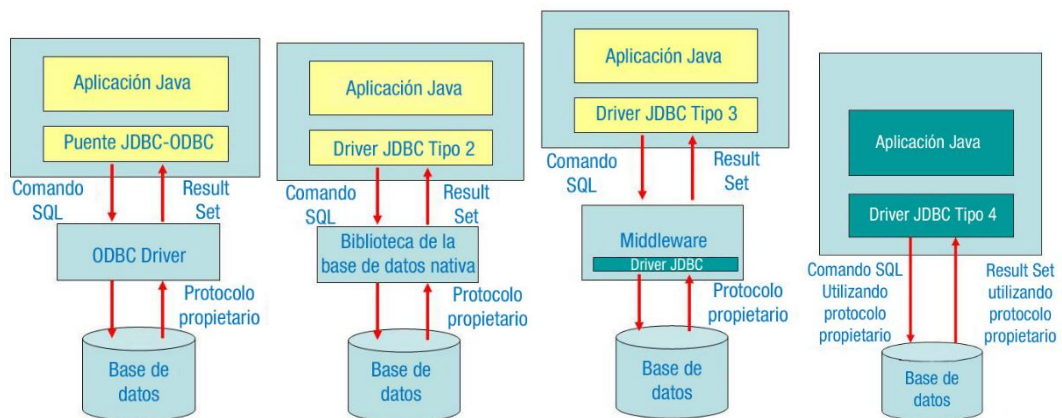
No requiere librerías específicas de la base de datos, lo que lo hace ideal para aplicaciones de **Internet**. Y el middleware centralizado permite funciones avanzadas como balanceo de carga y administración de sistemas (auditoría y monitorización).

TIPO 4 (PROTOCOLO NATIVO)

Este conector convierte directamente las llamadas JDBC al **protocolo** de red del gestor de base de datos, permitiendo una **comunicación directa** entre el cliente y el servidor. Es ideal para entornos **intranet**.

Elimina la capa del middleware y traducciones adicionales, mejorando el rendimiento. Además, no requiere instalar software especial en el cliente o en el servidor (el usuario necesita un driver específico para cada base de datos).

Un ejemplo es **Oracle Thin**.



UNIDAD 3: Herramientas de mapeo objeto-relacional (ORM).

UNIDAD 4: Bases de datos objeto-relacionales y orientadas a objetos.

Las **BDR (Bases de Datos Relacionales)** son adecuadas para datos simples y estructuras estables (incluso cuando la aplicación está desarrollada en un lenguaje orientado a objetos y sea necesario un ORM), pero presentan limitaciones con datos complejos o cambiantes: requieren dividir objetos en múltiples tablas, lo que genera numerosos joins y afecta el rendimiento.

Las **BDOO (Bases de Datos Orientadas a Objetos)** surgieron para superar estas limitaciones, integrando directamente objetos persistentes sin necesidad de mapeo ya que soportan un modelo de objetos puro, aunque no lograron imponerse por falta de estándar.

Las **BDOR (Bases de Datos Objeto-Relacionales)** combinan lo relacional con conceptos OO, mejorando capacidades, pero mantienen un mapeo subyacente costoso y poco flexible para estructuras muy complejas.

1. Características de las BDOO.

En una BDOO, los datos se almacenan como objetos con estado (atributos) y comportamiento (métodos), integrando el modelo de objetos de forma pura. Las entidades del modelo relacional equivalen a clases; las ocurrencias, a objetos instanciados; y las relaciones se gestionan por inclusión lógica, sin claves primarias, usando identificadores únicos (OID).

Características clave:

- Soporta agregación, encapsulamiento, polimorfismo y herencia (persistente en la BD).
- Identificador de Objeto (OID) único e independiente del contenido del objeto (aunque la información del objeto cambie, el OID sigue siendo el mismo). Dos objetos son equivalentes si tienen la misma información, pero distinto OID.
- Jerarquía y extensión de tipos.
- Objetos complejos con estructura arbitraria.
- Acceso navegacional, sin joins.
- Gestión de versiones de objetos.

VENTAJAS

- Ideales para aplicaciones con muchos tipos de datos, relaciones complejas y objetos con comportamiento elaborado.
- Transparencia → El programador solo gestiona objetos, no su almacenamiento físico.
- Gran capacidad de modelado (herencia, encapsulamiento).

- Flexibilidad estructural y extensibilidad (nuevos tipos a partir de existentes).
- Acceso rápido a objetos complejos mediante referencias, sin necesidad de reensamblarlos.
- Reutilización de código y menores costes de desarrollo.
- Control de acceso/concurrencia por objetos o jerarquías.
- Eficiencia en entornos distribuidos y cliente/servidor.

DESVENTAJAS

- Ausencia de un modelo de datos universal y base teórica consolidada.
- Falta de estándares generales.
- Mayor complejidad estructural frente a BDR.
- Fuerte competencia de BDR y BDOR, ya asentadas.
- Optimización de consultas difícil → Requiere conocer la implementación, lo que contradice el encapsulamiento.

2. Gestores de BDOO.

Un Sistema Gestor de Bases de Datos Orientadas a Objetos (SGBDOO, en inglés ODBMS) es un software que sirve de interfaz entre una base de objetos, los usuarios y las aplicaciones. Combina las funciones propias de un SGBD (acceso optimizado, concurrencia, seguridad, recuperación ante fallos) con el paradigma de orientación a objetos (identidad, encapsulación, herencia, polimorfismo y control de tipos), permitiendo almacenar objetos de forma sencilla. El estándar ODMG-93, propuesto por el grupo ODMG, fue un intento de normalización sin mucho éxito, pero sirvió como punto de partida.

El desarrollo de SGBDOO sigue tres enfoques principales:

- Ampliar lenguajes OO con capacidades de BD (GemStone).
- Añadir bibliotecas de clases que incorporan persistencia, transacciones y concurrencia (ObjectStore, Versant).
- Extender lenguajes de BD con capacidades OO (SQL 2003, OQL).

La falta de un estándar real limita la portabilidad e interoperabilidad, reduciendo la variedad de productos frente a las bases relacionales. Algunos ejemplos son: Db4o (open source, Java/.NET), **Matisse** (basado en ODMG, varios lenguajes), **ObjectDB** (Java, C++, Python, propietario con prueba), EyeDB (software libre, ODMG, C++/Java), Neodatis, ObjectStore y GemStone.

OBJETOS SIMPLES Y ESTRUCTURADOS

En las bases de datos orientadas a objetos (BDOO), los objetos se relacionan mediante referencias. Se distinguen dos tipos:

- **Objeto simple** → Estructura de un solo nivel, no contiene otros objetos.
- **Objeto estructurado** → Incluye componentes que pueden ser otros objetos, aplicando constructores recursivamente.

Las referencias entre objetos pueden ser:

- **Referencia de propiedad (es-parte-de)** → El componente está encapsulado dentro del objeto, no tiene identificador propio, solo es accesible por sus métodos y se elimina con el objeto contenedor.
- **Referencia de asociación (está-asociado-con)** → El componente es un objeto independiente, con identidad propia, accesible mediante sus métodos y no se elimina automáticamente al eliminar el objeto que lo referencia.

Las referencias de asociación equivalen a las relaciones del modelo relacional (claves foráneas, 1:1, 1:N, N:M). Una relación 1:N se implementa con una colección (List, Set, etc.), que es un objeto en sí mismo y puede recuperarse o almacenarse junto con su contenedor en una sola operación.

3. El API de la base de datos.

Los SGBDOO ofrecen una API común para lenguajes orientados a objetos, que permite interactuar con la base de datos. Esta API suele incluir:

- Contenedor de objetos → Interfaz principal para operar con la base (guardar, consultar, borrar, cerrar, transacciones).
- Modos de trabajo → Embebido (local) o cliente-servidor.
- Resultados de consultas → Devueltos como conjuntos de objetos.
- Configuración → Personalización del comportamiento de la base, normalmente antes de abrir la sesión.
- Consultas → Mecanismos para definir condiciones de búsqueda.

Siempre se opera a través del contenedor de objetos, que es la puerta de entrada a la base.

APERTURA Y CIERRE DE CONEXIONES

En los SGBDOO, la conexión desde una aplicación se hace mediante la API propia del gestor.

El flujo general de trabajo con una base de objetos es:

- **Abrir conexión** → Se crea o accede a la base de datos. En modo local solo se permite una conexión; en modo cliente-servidor se pueden abrir múltiples conexiones.
- **Realizar operaciones** → Se ejecutan las acciones sobre los objetos: almacenar, consultar, modificar, borrar, etc.
- **Cerrar conexión** → Se libera el recurso cerrando la conexión con la base.

Dado que abrir nuevas conexiones es más costoso, suele interesar más usar un **pool de conexiones** para reutilizar conexiones ya abiertas.

4. El lenguaje de consulta de objetos OQL.

OQL (Object Query Language) es el lenguaje de consulta propuesto por el estándar ODMG para bases de datos orientadas a objetos.

Características principales:

- Lenguaje declarativo, similar a SQL, pero adaptado al modelo de objetos (identidad, herencia, polimorfismo, relaciones, objetos complejos).
- No incluye operaciones de modificación, los cambios se realizan mediante métodos de los objetos.
- Puede usarse de forma autónoma o incrustada en lenguajes como C++, Java o Smalltalk.
- Las consultas incrustadas pueden devolver objetos compatibles con el sistema de tipos del lenguaje anfitrión.
- Permite invocar métodos escritos en esos lenguajes.
- Soporta dos formas de acceso: asociativo (devuelve colecciones de objetos) y navegacional (accede a objetos individuales recorriendo sus interrelaciones).

CONSULTAS SELECT

- ▶ Terminan siempre en punto y coma.
- ▶ Incluyen cláusulas como WHERE, GROUP BY, ORDER BY, AS (alias) y LIKE (patrones).
- ▶ SELECT * devuelve el OID de cada objeto (hexadecimal) y sus interrelaciones, aunque estas solo muestran el primer objeto relacionado, incluso si la relación es múltiple.
- ▶ Se pueden hacer JOIN entre clases para consultas asociativas.
- ▶ Permite consultas navegacionales:
 - Accediendo directamente a las interrelaciones.
 - Mediante REF() para navegar por referencias a objetos.

```
SELECT [DISTINCT] <expresión, ...>  
FROM <lista from>  
[WHERE <condición>]  
[ORDER BY <expresión>]
```

```
SELECT p.nombre, p.email  
FROM p in Profesor  
WHERE p.ingreso <= 1990  
ORDER BY p.nombre;
```

```
SELECT p.nombre, t.titulo, t.tema  
FROM Profesor p  
JOIN Tesis t  
ON t.es_dirigida = p.OID;
```

5. Características de las BDOR.

Las bases de datos objeto-relacionales (BDOR) combinan el modelo relacional con la orientación a objetos, buscando mejorar la representación de los datos y simplificar su acceso. Aunque siguen usando tablas con filas y columnas, estas pueden contener tipos estructurados (no solo valores atómicos) y permiten herencia entre tablas. Esto es posible porque tanto las tablas como las columnas se tratan internamente como objetos, mediante un mapeo objeto-relacional transparente.

Entre sus características principales están:

- Tipos definidos por el usuario (compuestos o multivaluados).
- Tipos objeto, que permiten relaciones anidadas.
- Reutilización de tipos en distintas tablas.
- Funciones y métodos almacenados en el gestor.
- Tablas anidadas (arrays o vectores).
- Herencia de subtipos y subtablas.

Estas capacidades están recogidas en el estándar **SQL:1999**.

6. Gestores de BDOR.

Los Sistemas Gestores de Bases de Datos Objeto-Relacionales (SGBDOR) fusionan la tecnología relacional con la de objetos, manteniendo las bases relacionales, pero con restricciones. Ejemplos de estos gestores son Oracle, PostgreSQL y MySQL.

El paradigma orientado a objetos supone un avance al combinar datos y procedimientos en una misma entidad, lo que permite la reutilización y un comportamiento predecible. Sin embargo, las bases de datos orientadas a objetos puras tienen desventajas por lo que no reemplazarán completamente a las relacionales.

Los objetos se integran en las bases de datos relacionales de dos formas principales:

- **Como dominios** → Actuando como tipo de dato de una columna, lo que permite almacenar múltiples valores en una celda (aunque sigue siendo un solo objeto por fila) y asociar procedimientos (métodos) a esos datos.
- **Como tablas de objetos** → Donde cada fila de la tabla es un objeto completo.

Un ORDBMS (Sistema Gestor de Bases de Datos Relacional con Orientación a Objetos) es un sistema relacional que incorpora características de objetos (tipos definidos por el usuario, herencia, polimorfismo), pero sigue sujeto a las restricciones relacionales y conserva la capacidad de realizar operaciones join.

TIPOS DE OBJETOS

Un Sistema Gestor de Bases de Datos Objeto-Relacional (SGBDOR) extiende el modelo relacional permitiendo crear tipos de objeto complejos, que encapsulan datos y operaciones.

Un **tipo de objeto** es una plantilla que representa una entidad real y se compone de:

- **Nombre** → Identifica al tipo.
- **Atributos** → Modelan la estructura y datos (obligatorios). Pueden ser de tipo básico o definido por el usuario.
- **Métodos** → Subprogramas (procedimientos o funciones) que definen el comportamiento (opcionales). Se escriben en PL/SQL o C.

```
CREATE TYPE direccion_t AS OBJECT (  
  calle VARCHAR2(200),  
  ciudad VARCHAR2(200),  
  prov CHAR(2),  
  codpos VARCHAR2(20));
```

```
CREATE TYPE cliente_t AS OBJECT (  
  clinum NUMBER,  
  clinomb VARCHAR2(200),  
  direccion direccion_t, -- Uso de tipo creado  
    anteriormente  
  telefono VARCHAR2(20),  
  fecha_nac DATE,  
  MEMBER FUNCTION edad RETURN NUMBER,  
  PRAGMA  
  RESTRICT_REFERENCES(edad,WNDS));
```

Características de los **atributos**:

- Se declaran con un nombre (único dentro del tipo) y un tipo de datos.
- No pueden inicializarse con DEFAULT ni tener restricción NOT NULL en su declaración (aunque sí pueden aplicarse estas restricciones al almacenar el objeto en una tabla).

Características de los **métodos**:

- Se declaran con la palabra clave **MEMBER** y no pueden coincidir con el nombre del tipo ni de sus atributos.
- Constan de una especificación (firma) y un cuerpo (código ejecutable), que deben coincidir exactamente.
- Pueden acceder directamente a los atributos y otros métodos del mismo tipo sin cualificadores, y se ejecutan sobre objetos de su tipo.

```
-- Creación del método getEdad de la clase  
  cliente_t (Create type)  
CREATE OR REPLACE TYPE BODY cliente_t AS  
  MEMBER FUNCTION getEdad RETURN NUMBER IS  
    a NUMBER;  
    d DATE;  
  BEGIN  
    d:= today();  
    a:= d.año - fecha_nac.año;  
    IF (d.mes < fecha_nac.mes) OR ((d.mes =  
    fecha_nac.mes) AND (d.día < fecha_nac.día))  
    THEN  
      a:= a-1;  
    END IF;  
    RETURN a;  
  END;  
END;
```

7. Gestión de transacciones en las BDOR.

En un sistema de bases de datos objeto-relacional, una transacción es una unidad de trabajo indivisible (atómica): o se ejecutan todas las operaciones que la componen o no se ejecuta ninguna.

Su objetivo es garantizar el acceso concurrente, la integridad y la recuperación ante fallos. Para ello, deben cumplir las **propiedades ACID**:

- **Atomicidad** → Todo o nada.
- **Consistencia** → La BD queda en un estado válido al finalizar.
- **Aislamiento (Isolation)** → Las transacciones concurrentes no interfieren entre sí.
- **Durabilidad** → Una vez completada, el cambio es permanente.

Además, se pueden usar puntos de salvaguarda (**savepoints**) para deshacer solo una parte de la transacción, permitiendo retroceder a un punto intermedio sin anularla por completo. Esta característica, originaria del estándar SQL99 (objeto-relacional), se ha adoptado también en bases de datos relacionales como MySQL con InnoDB.

UNIDAD 5: Bases de datos NoSQL.

Existen **tres tipos de datos** según su nivel de estructuración: **estructurados** (formato estricto, como tablas), **desestructurados** (sin estructura, como texto o video) y **semi-estructurados** (estructura variable y dinámica).

Las bases de datos relacionales tradicionales son ideales para datos estructurados, pero surge la necesidad de manejar grandes volúmenes de datos no estructurados e integrarlos. Para ello entra en juego **XML (eXtensible Markup Language)**, un estándar del **W3C** que organiza la información mediante reglas semánticas.

Sus principales **ventajas** son: ser un lenguaje bien formado, extensible, fácil de leer, autodescriptivo e intercambiable entre arquitecturas, requiriendo un **parser** para su interpretación.

Gracias a estas ventajas, XML se adoptó masivamente para el almacenamiento e intercambio de información, lo que creó la necesidad de almacenar y recuperar eficientemente estos datos. Para lograrlo, y dependiendo del tipo de documento XML (centrado en datos o en texto), surgen las **Bases de Datos XML**, cuyo objetivo es almacenar y consultar datos poco estructurados con la eficiencia de las bases de datos convencionales.

Existen dos tipos principales de documentos XML:

- **Documentos centrados en datos** → Se caracterizan por tener muchos elementos pequeños, una estructura regular y bien definida, y estar dirigidos al procesamiento automático por máquinas (ej. facturas, pedidos, fichas de alumno).
- **Documentos centrados en texto** → Tienen pocos elementos, grandes cantidades de texto y estructuras impredecibles. Son datos poco estructurados, orientados a la lectura humana y típicos de sistemas documentales (ej. libros, informes, artículos).

Existen tres opciones principales para **almacenar documentos XML**:

- ▶ **Almacenamiento directo del fichero** → Es limitado, ya que solo permite las operaciones básicas del sistema de archivos.
- ▶ **Base de datos existente (relacional, orientada a objetos, etc.):**
 - Como BLOB → Se guarda el documento intacto. Es fácil de implementar y útil para contenido estático, pero dificulta las consultas.
 - Mediante mapeo → Se transforma el documento para ajustarse a la estructura de la base de datos. El inconveniente es que se puede perder información y el documento no se recupera de forma intacta, siendo casi imposible para documentos centrados en texto.
- ▶ **Base de datos nativa XML** → Es la mejor opción, especialmente para documentos basados en texto, ya que el documento se almacena y recupera siempre de forma intacta.

En la actualidad, los sistemas que soportan XML se dividen en dos tipos principales, cuya diferencia clave es que las **BD Nativas** usan un modelo propio y manejan cualquier documento, mientras que las **BD Compatibles** añaden una capa de software sobre un modelo existente y solo manejan documentos que se ajustan a él.

1. Bases de Datos Nativas XML.

Una **Base de Datos Nativa XML (NXD, Native XML Database)** se define por tres propiedades fundamentales:

- Define un modelo lógico para almacenar y recuperar el documento XML de forma intacta (incluyendo elementos, atributos, texto y orden).
- El documento XML es la unidad lógica mínima de almacenamiento.
- No depende de un modelo de almacenamiento físico concreto, pudiendo construirse sobre diversas tecnologías.

Su diferencia clave frente a las bases de datos convencionales es que están especializadas en manejar documentos XML con todos sus componentes.

Otras características importantes incluyen: agrupar documentos en colecciones, permitir indexación, asignar un identificador único, soportar lenguajes de consulta (como XQuery) y actualización (Update XQuery), y no requerir validación previa (basta con que el documento esté bien formado). Además, soportan transacciones y control de accesos.

Su uso es imprescindible cuando hay documentos con anidamientos profundos, se necesita preservar la integridad total del documento o se realizan búsquedas de contenido frecuentes.

ESTRATEGIAS DE ALMACENAMIENTO

Las bases de datos nativas XML se pueden clasificar según su tipo de almacenamiento en dos estrategias principales, además de soluciones propietarias específicas.

- ▶ **Almacenamiento basado en texto** → Guarda el documento XML completo como texto. Suele usar compresión e índices para mejorar el acceso. Existen dos variantes:
 - Como un BLOB en una base de datos relacional o un fichero, con índices adicionales.
 - En un almacén especializado con soporte para transacciones e indexación.
- ▶ **Almacenamiento basado en modelo** → Define un modelo lógico (como DOM) para la estructura jerárquica del XML y almacena una representación binaria de ese modelo. Las opciones son:
 - Traducir el modelo a tablas relacionales.

- Traducir el modelo a objetos en una base de datos orientada a objetos.
- Usar un almacén creado específicamente para este fin.

Ventajas principales de estas bases de datos:

- No requieren mapeos adicionales para guardar los documentos.
- Conservan la integridad del documento original.
- Permiten almacenar documentos heterogéneos, con diferentes estructuras.

COLECCIONES Y DOCUMENTOS

Una base de datos XML se organiza en una estructura jerárquica con un nodo raíz que contiene colecciones y documentos.

- **Colección** → Agrupa documentos por su información y puede contener otras subcolecciones.
- **Documento** → Puede contener tanto información XML como datos de otro tipo (non-XML data).

Haciendo una analogía, las colecciones equivalen a las tablas en una base de datos relacional o a los directorios en un sistema de archivos, mientras que los documentos equivalen a las filas o a los ficheros.

Aunque varía según la implementación, las colecciones permiten gestionar la estructura de forma flexible: pueden tener múltiples DOCTYPEs (que incluso se crean dinámicamente si el documento no tiene uno) y esquemas asociados con definiciones lógicas y físicas. Además, existe un DOCTYPE especial llamado **non-XML** para almacenar documentos que no están en formato XML.

GESTORES NATIVOS XML COMERCIALES Y LIBRES

Existen gestores de bases de datos nativas XML tanto comerciales (código propietario) como libres (código abierto).

Gestores comerciales o propietarios:

- TaminoXML Server (SoftwareAG) → Producto de alto rendimiento que almacena documentos en una base propia sin transformarlos, separa documentos e índices, y soporta XQuery y APIs para Java, C y .NET.
- TEXTML (Isiasoft) → Almacena documentos en formato nativo sin mapeo, no requiere DTD o esquema, indexa según la estructura del documento, permite múltiples índices simultáneos e incluye APIs para Java, .NET y WebDAV.

Gestores libres o de código abierto:

- eXist → Usa un sistema de almacenamiento propio basado en árboles B+. Se ejecuta como servidor independiente, biblioteca Java embebida o en un motor servlet. Soporta colecciones jerárquicas, no requiere esquema previo y trabaja con XQuery y XUpdate.
- MongoDB → Base de datos NoSQL orientada a documentos de código abierto. Almacena los datos en estructuras BSON (similares a JSON) con un esquema dinámico, no en tablas.

2. Base de datos eXist.

eXist-db es un sistema de gestión de bases de datos de código abierto, clasificado como **base de datos NoSQL orientada a documentos y nativa XML**, que soporta documentos **XML, JSON, HTML y binarios**.

Su principal diferencia frente a las bases de datos relacionales es que utiliza **XQuery y XSLT** como lenguajes de consulta y programación. Permite conservar estos documentos sin necesidad de middleware extenso, sigue y amplía estándares del W3C como XQuery, y ofrece interfaces REST para interactuar con aplicaciones web.

LENGUAJE DE CONSULTA XPATH

XPath es un lenguaje estándar del W3C para navegar y seleccionar partes de un documento XML usando rutas, similar a cómo las expresiones regulares trabajan con texto plano. Es fundamental en tecnologías como XSLT y XQuery.

Sus características principales son: usar expresiones de ruta para navegar, tener una biblioteca de funciones estándar y permitir seleccionar nodos concretos del árbol XML.

El documento se modela como un árbol de 7 tipos de nodos (raíz, elemento, atributo, texto, etc.). La navegación se realiza desde un nodo de contexto usando ejes (como child, parent o ancestor) que definen la dirección de búsqueda. Las rutas de localización pueden ser absolutas o relativas, y se pueden escribir con una sintaxis completa o con una versión más corta y práctica llamada sintaxis abreviada.

LENGUAJE DE CONSULTAS XQUERY

XQuery es el lenguaje de consulta estándar para bases de datos XML, equivalente en su función a lo que SQL representa para las bases de datos relacionales, pero adaptado a datos jerárquicos donde el orden y la estructura arbórea son fundamentales.

Se basa en XPath para la navegación y es de tipo declarativo (se describe qué resultado se quiere, no cómo obtenerlo).

Sus expresiones más importantes son:

- Expresiones XPath, para navegar y seleccionar nodos.
- Expresiones FLWOR (For, Let, Where, Order, Return), para iterar y transformar datos de forma potente, similares al SELECT de SQL.

Además, incorpora condicionales, cuantificadores y funciones de agregación para construir resultados complejos.

3. Bases de Datos MongoDB.

MongoDB es la **base de datos NoSQL** líder del mercado, diseñada para ofrecer agilidad y escalabilidad a empresas de todos los tamaños.

Sus características principales son:

- **Esquema dinámico** → Permite que la estructura de los datos evolucione rápidamente sin interrumpir las aplicaciones.
- **Alto rendimiento** → Creada para lectura y escritura rápidas, potenciando la computación en memoria.
- **Fiabilidad empresarial** → Su replicación nativa y tolerancia a fallos automática garantizan la continuidad del servicio y la flexibilidad operativa.

Además de la versión de código abierto, MongoDB ofrece suscripciones comerciales (MongoDB Enterprise) que brindan características avanzadas de seguridad, monitorización, certificaciones y soporte técnico profesional. Su servicio de gestión (MMS) añade funcionalidades de monitorización y copias de seguridad tanto en la nube como localmente.